Programming Principles I

Week 7 · Class 13 · Quiz 3

Quiz 3: 25 minutes

Pen only. No laptop, no phone. Write your name and student ID first.

Programming Principles I

Week 7 · Class 13

Two-dimensional arrays

Tables of numbers · rows and columns · two loops, one inside the other

```
int t[3][4];
t[1][2] = 9;
```

What you know about arrays

1. `int day[4];` makes 4 boxes: `day[0]` to `day[3]`
2. `int day[4] = {3, 5, 8, 4};` fills them
3. A `for` loop visits every box: `c < 4`, never `c <= 4`
4. C does not check the index: you do

Recap: what is on the screen?

PREDICT

One day, four temperatures.

What does it print?

```
int main(void) {
    int day[4] = {3, 5, 8, 4};
    for (int c = 0; c < 4; c++) {
        printf("%d ", day[c]);
    }
    printf("\n");
    return 0;
}
```

Recap: what is on the screen?

PREDICT

One day, four temperatures.

What does it print?

```c
int main(void) {
    int day[4] = {3, 5, 8, 4};
    for (int c = 0; c < 4; c++) {
        printf("%d ", day[c]);
    }
    printf("\n");
    return 0;
}
```

ANSWER

3 5 8 4

c goes 0, 1, 2, 3.

One row is not enough

Three days, four times a day: 12 numbers.

Twelve variables? No.

Three arrays `day0`, `day1`, `day2`? Better, but no.

One **table**: 3 rows, 4 columns.

Today's roadmap

1. A **table**: `int t[3][4];`
2. **Two loops**, one inside the other
3. **Row sums** and **column sums**
4. **Read** a table, and two **careful** cases
5. **Your turn**, and **Project 2**

1 A table

Rows first, then columns

int t[3][4];

3 rows, 4 columns: 12 `int` boxes.

The first number: how many **rows**.

The second number: how many **columns**.

One box: `t[row][column]`. Both start at 0.

The table t

	column 0	column 1	column 2	column 3
row 0	3	5	8	4
row 1	2	6	**9**	1
row 2	-1	0	4	2

`t[1][2]` is row 1, column 2: **9**. Rows go 0 to 2, columns 0 to 3.

Fill it with a list

```
int t[3][4] = {
    {3, 5, 8, 4},
    {2, 6, 9, 1},
    {-1, 0, 4, 2}
};
printf("%d\n", t[0][0]);
printf("%d\n", t[1][2]);
printf("%d\n", t[2][3]);
```

One pair of braces per row.

Rows are separated by commas.

Predict: three boxes

PREDICT

What do the three `printf` lines print?

```
int t[3][4] = {
    {3, 5, 8, 4},
    {2, 6, 9, 1},
    {-1, 0, 4, 2}
};
printf("%d\n", t[0][0]);
printf("%d\n", t[1][2]);
printf("%d\n", t[2][3]);
```

Predict: three boxes

PREDICT

What do the three `printf` lines print?

```
int t[3][4] = {
    {3, 5, 8, 4},
    {2, 6, 9, 1},
    {-1, 0, 4, 2}
};
printf("%d\n", t[0][0]);
printf("%d\n", t[1][2]);
printf("%d\n", t[2][3]);
```

ANSWER

3, 9, 2

The first box, row 1 column 2, and the last box.

Careful: the order

CAREFUL

Is `t[1][2]` the same box as `t[2][1]`?

Careful: the order

CAREFUL

Is `t[1][2]` the same box as `t[2][1]`?

ANSWER

No. `t[1][2]` is row 1, column 2: **9**.
`t[2][1]` is row 2, column 1: **0**. Row first, always.

Missing values become 0

```
int t[2][3] = {
    {1},
    {4, 5}
};
printf("%d %d %d\n", t[0][0], t[0][1], t[0][2]);
printf("%d %d %d\n", t[1][0], t[1][1], t[1][2]);
```

Output

```
1 0 0
4 5 0
```

`= {0}` makes every box 0.

2 Two loops

One loop inside the other

Visit every box

```c
int main(void) {
    for (int r = 0; r < 3; r++) {
        for (int c = 0; c < 4; c++) {
            printf("(%d,%d) ", r, c);
        }
        printf("\n");
    }
    return 0;
}
```

Output

```
(0,0) (0,1) (0,2) (0,3)
(1,0) (1,1) (1,2) (1,3)
(2,0) (2,1) (2,2) (2,3)
```

Outer loop: the row.

Inner loop: every column.

The inner loop runs to the end first

For every row, all 4 columns.

`r` is 0: `c` goes 0, 1, 2, 3. Then `r` is 1: `c` starts again at 0.

3 rows times 4 columns: the inner line runs **12** times.

After `(0,3)` comes `(1,0)`.

Print the whole table

```
for (int r = 0; r < 3; r++) {
    for (int c = 0; c < 4; c++) {
        printf("%4d", t[r][c]);
    }
    printf("\n");
}
```

Output

```
   3   5   8   4
   2   6   9   1
  -1   0   4   2
```

%4d: 4 places, the columns line up.

A new line after each row.

Write the sizes once: #define

```c
#define ROWS 3
#define COLS 4

int main(void) {
    int t[ROWS][COLS] = {
        {3, 5, 8, 4},
        {2, 6, 9, 1},
        {-1, 0, 4, 2}
    };
```

ROWS and COLS are names for 3 and 4.

Change one line, every loop follows.

Labels on the board

```
printf("    ");
for (int c = 0; c < COLS; c++) {
    printf("%4d", c);
}
printf("\n");
for (int r = 0; r < ROWS; r++) {
    printf("%d | ", r);
    for (int c = 0; c < COLS; c++) {
        printf("%4d", t[r][c]);
    }
    printf("\n");
}
```

Output

```
      0   1   2   3
0 |   3   5   8   4
1 |   2   6   9   1
2 |  -1   0   4   2
```

Column numbers on top, the row number in front.

Fill it with a loop

```
int m[ROWS][COLS];
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        m[r][c] = (r + 1) * (c + 1);
    }
}
```

Output

```
 1   2   3   4   5
 2   4   6   8  10
 3   6   9  12  15
 4   8  12  16  20
```

ROWS 4, COLS 5.

Predict: one box

PREDICT

The highlighted line fills every box.

What is `m[2][3]`?

```
int m[ROWS][COLS];
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        m[r][c] = (r + 1) * (c + 1);
    }
}
```

Predict: one box

PREDICT

The highlighted line fills every box.

What is `m[2][3]`?

```
int m[ROWS][COLS];
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        m[r][c] = (r + 1) * (c + 1);
    }
}
```

ANSWER

12: (2 + 1) * (3 + 1).
Row 2, column 3 of the output.

3

Row and column sums

Where the sum starts matters

The sum of every row

```
for (int r = 0; r < ROWS; r++) {
    int sum = 0;
    for (int c = 0; c < COLS; c++) {
        sum += t[r][c];
    }
    printf("row %d: %d\n", r, sum);
}
```

Output

```
row 0: 20
row 1: 18
row 2: 5
```

`sum = 0` for every row.

Row 0, step by step

```
for (int r = 0; r < ROWS; r++) {
    int sum = 0;
    for (int c = 0; c < COLS; c++) {
        sum += t[r][c];
    }
    printf("row %d: %d\n", r, sum);
}
```

c	t[0][c]	sum
-	-	0
0	3	3
1	**5**	**8**

Row 0, step by step

```
for (int r = 0; r < ROWS; r++) {
    int sum = 0;
    for (int c = 0; c < COLS; c++) {
        sum += t[r][c];
    }
    printf("row %d: %d\n", r, sum);
}
```

c	t[0][c]	sum
-	-	0
0	3	3
1	5	8
2	8	16
3	**4**	**20**

Then `printf`: `row 0: 20`. Row 1 starts again at 0.

Careful: where does sum start?

CAREFUL

Move `int sum = 0;` above the outer loop.

What do rows 1 and 2 print now?

Careful: where does sum start?

CAREFUL

Move `int sum = 0;` above the outer loop.

What do rows 1 and 2 print now?

ANSWER

`row 1: 38` and `row 2: 43`.

The sum never starts again: 20 + 18, then 38 + 5.

The sum of every column

```
for (int c = 0; c < COLS; c++) {
    int sum = 0;
    for (int r = 0; r < ROWS; r++) {
        sum += t[r][c];
    }
    printf("column %d: %d\n", c, sum);
}
```

Output

```
column 0: 4
column 1: 11
column 2: 21
column 3: 7
```

Outer loop: the column.

Rows or columns: swap the loops

sum of every	outer loop	inner loop	the box
row	`r < ROWS`	`c < COLS`	`t[r][c]`
column	`c < COLS`	`r < ROWS`	`t[r][c]`

The box is still `t[r][c]`: row first. Only the order of the loops changes.

4 Read, and be careful

scanf into a table · indexes C does not check

Read a table with scanf

```
int t[ROWS][COLS];
printf("Type %d numbers:\n", ROWS * COLS);
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        scanf("%d", &t[r][c]);
    }
}
```

Output

```
Type 6 numbers:
1 2 3
4 5 6
   1   2   3
   4   5   6
```

`&t[r][c]`: the `&`, as for any `int`.

Predict: column 4

PREDICT

Columns go 0 to 3.

What does `t[0][4]` do?

```
int t[3][4] = {
    {3, 5, 8, 4},
    {2, 6, 9, 1},
    {-1, 0, 4, 2}
};
printf("%d\n", t[0][4]);
```

It prints 2

```
int t[3][4] = {
    {3, 5, 8, 4},
    {2, 6, 9, 1},
    {-1, 0, 4, 2}
};
printf("%d\n", t[0][4]);
```

Output

```
2
```

No error. No warning.

2 is `t[1][0]`.

The rows sit one after another

[0][0]	[0][1]	[0][2]	[0][3]	[1][0]	[1][1]	…
3	5	8	4	**2**	6	…

In memory, row 1 comes right after row 0. One box past the end of row 0 is the first box of row 1.

One row too far

```
for (int r = 0; r <= ROWS; r++) {
    printf("%d ", t[r][0]);
}
printf("\n");
```

Two runs

```
3 2 -1 943716464

3 2 -1 -136316352
```

There is no row 3.

gcc says nothing.

The rule for indexes

Rows `0` to `ROWS - 1`. Columns `0` to `COLS - 1`.

Loops: `r < ROWS` and `c < COLS`. Never `<=`.

C does not check. A wrong index reads some other box, or garbage.

A table of char

```
char g[ROWS][COLS];
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        g[r][c] = '.';
    }
}
g[1][3] = '*';
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        printf("%c ", g[r][c]);
    }
    printf("\n");
}
```

Output

```
. . . . .
. . . * .
. . . . .
```

`g[1][3]`: row 1, column 3.

%c for one character.

5

Your turn

Two short tasks on paper

Your turn: below zero

YOUR TURN

Use the table `t` of today.

Count the boxes below 0.

Print the row and column of each one.

Below zero: one answer

```
int count = 0;
for (int r = 0; r < ROWS; r++) {
    for (int c = 0; c < COLS; c++) {
        if (t[r][c] < 0) {
            printf("below zero at (%d,%d)\n", r, c);
            count++;
        }
    }
}
printf("%d below zero\n", count);
```

Output: `below zero at (2,0)`, then `1 below zero`. The count starts once, before both loops.

Your turn: the largest of each row

YOUR TURN

Print the largest value of every row of `t`.

Use the pattern of class 10.

Largest of each row: one answer

```
for (int r = 0; r < ROWS; r++) {
    int max = t[r][0];
    for (int c = 1; c < COLS; c++) {
        if (t[r][c] > max) {
            max = t[r][c];
        }
    }
    printf("row %d: max %d\n", r, max);
}
```

Output

```
row 0: max 8
row 1: max 9
row 2: max 4
```

`max` starts at the first box of the row.

Today in one slide

1. `int t[3][4];`: 3 rows, 4 columns, 12 boxes
2. `t[r][c]`: row first, then column; both from 0
3. Two loops: the outer one picks the row
4. Row sums: `sum = 0` inside the outer loop
5. `r < ROWS`, `c < COLS`: C does not check

Project 2: Treasure Hunt

A hidden 8 × 8 board: find 5 treasures, avoid 3 traps.

Two tables: what is in each cell, and what the player has seen.

Today's loops print the board. Friday: the cells around a cell.

Your own program, then a live talk with a TA about it.

Project 2: what and when

1. The brief: Blackboard, **Projects** folder
2. One file: `treasure_hunt.c`
3. Due **Monday 9 November, 23:59**
4. 50 points: the program · 50 points: the live talk

> Start early. Read the whole brief this week.

Before Friday

1. **Knowledge check 13** on Blackboard: today's slides
2. **Quiz 4: Fri 23 October**, classes 11-12: strings
3. **Midterm: Fri 30 October**, classes 1 to 14
4. Today's code: Blackboard, Week 7 module, **Class 13 code**

> Friday a TA teaches: tables in functions, matrices, the cells around a cell.